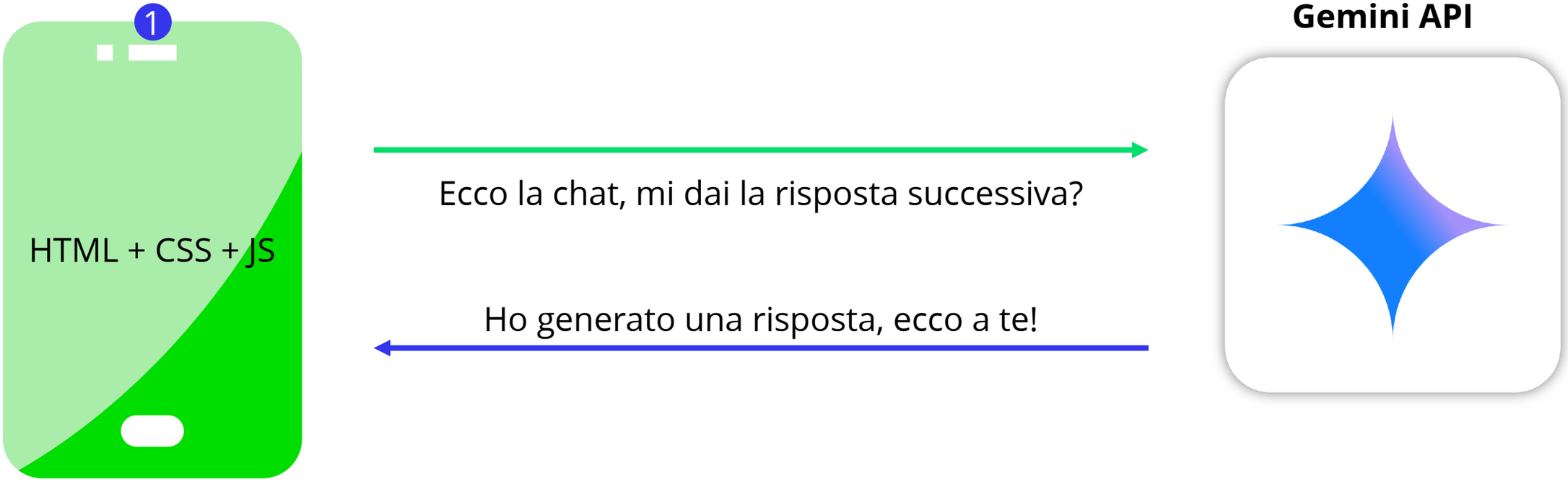

boolean

Il nostro obiettivo

Sfruttare un servizio esterno alla nostra app!





Gemini API

Diamo un'occhiata all'API messa a disposizione da Google

Gemini Web API

Servizio di chat con intelligenza artificiale

Una chat con l'intelligenza artificiale che possiamo usare nelle nostre App!
Ha una interfaccia web, ma mette a disposizione anche una API con la sua **documentazione**.
https://ai.google.dev/api/generate-content?hl=it

Ci sono 3 elementi fondamentali che dobbiamo reperire sulla documentazione di un'API:

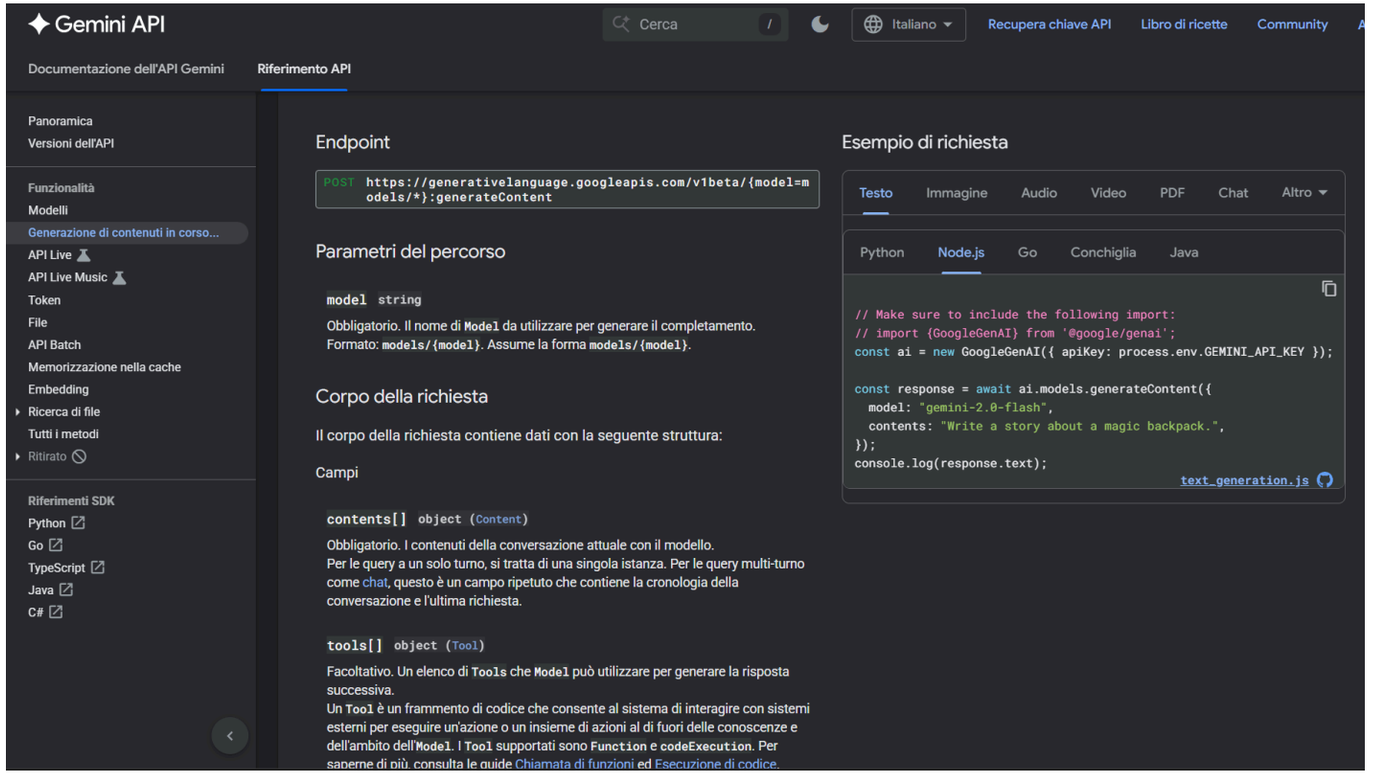
- L'**endpoint** a cui effettuare la chiamata
- L'**API Key** per identificarci, se necessario
- Il **Formato** che l'API si aspetta di ricevere



Gemini Web API

Endpoint

L'endpoint è l'indirizzo al quale dovremo effettuare la chiamata per ottenere le risposte da Gemini.
https://ai.google.dev/api/generate-content?hl=it#endpoint



Gemini Web API

API key

Si tratta di una stringa "segreta" che ci identifica e che fa capire a Google chi effettua la richiesta a Gemini.

Serve a tutelare gli utilizzi dell'API: controlla chi può accedere, limita l'uso per evitare abusi e protegge le risorse e i dati del servizio dagli accessi non autorizzati

Per generarne una, basta creare un account gratuito su Google AI Studio: https://aistudio.google.com/

Una volta ottenuta, possiamo "agganciarla" all'endpoint.




```
1 'https://example-endpoint?key=439582ir94u403u'
```

Gemini Web API

Formato

Il formato è fondamentale per capire con che struttura mandare i dati ad un'API.

Ogni API definisce nella propria documentazione la **struttura appropriata** per il proprio funzionamento

Nel caso della generazione di testo, Gemini si aspetta una struttura dati ben precisa:

https://ai.google.dev/api/generate-content?hl=it#content

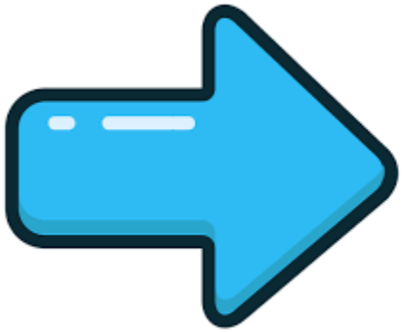
```
{
  "parts": [
    {
      object (Part)
    }
  ],
  "role": string
}
```

Gemini Web API

Formato

Il nostro array di messaggi andrà quindi trasformato di conseguenza:

```
const messages = [
  {
    type: 'sent',
    text: 'Ciao, come va?',
    time: '11/11/2025 13:23:00'
  },
  {
    type: 'received',
    text: 'tutto bene, tu?',
    time: '11/11/2025 13:25:00'
  },
]
```



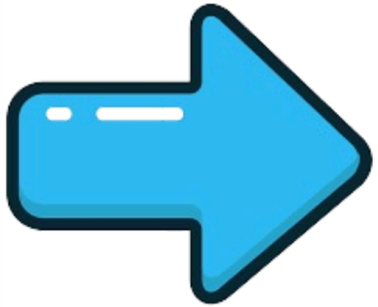
```
const formattedMessages = [
  {
    role: 'user',
    parts: [{ text: 'Ciao, come va?'}]
  },
  {
    role: 'model',
    parts: [{text: 'tutto bene, tu?'}]
  },
]
```

Gemini Web API

Formato

Il nostro array di messaggi andrà quindi trasformato di conseguenza:

```
const messages = [
  {
    type: 'sent',
    text: 'Ciao, come va?',
    time: '11/11/2025 13:23:00'
  },
  {
    type: 'received',
    text: 'tutto bene, tu?',
    time: '11/11/2025 13:25:00'
  },
]
```



```
const formattedMessages = [
  {
    role: 'user',
    parts: [{ text: 'Ciao, come va?'}]
  },
  {
    role: 'model',
    parts: [{text: 'tutto bene, tu?'}]
  },
]
```



e la data?

e il type?



Non importa passare data e tipo, perché all'AI interessano soltanto i messaggi di testo per poter inventare una risposta pertinente!

Inoltre, grazie al `role`, sa già chi ha detto cosa!

Costrutti condizionali

Operatore ternario

L'operatore ternario è una soluzione sintattica comoda quando dobbiamo assegnare un valore ad una variabile e abbiamo due opzioni tra cui scegliere.

Inizialmente facciamo un controllo, se la condizione è vera assegniamo il primo valore, altrimenti il secondo.

```
1 const age = 21;
2 const maggiorenne = age > 17 ? 'Sì' : 'No'
3
4 // Maggiorenne sarà 'Sì'
```

Gemini Web API

System prompt

Un ultimo passaggio prima di terminare: l'AI deve sapere come comportarsi, quindi all'inizio della conversazione, inseriamo il nostro system prompt!

```
// preparo le istruzioni iniziali per l'AI
  const SYSTEM_PROMPT = "Sei un'amica che risponde in modo amichevole e informale....";

  // Inseriamole per prime nell'array, sempre con lo stesso formato
  const formattedMessages = [
    {
      role: 'user',
      parts: [{ text: SYSTEM_PROMPT }]
    },
    // tutto il resto della chat...
  ];


```

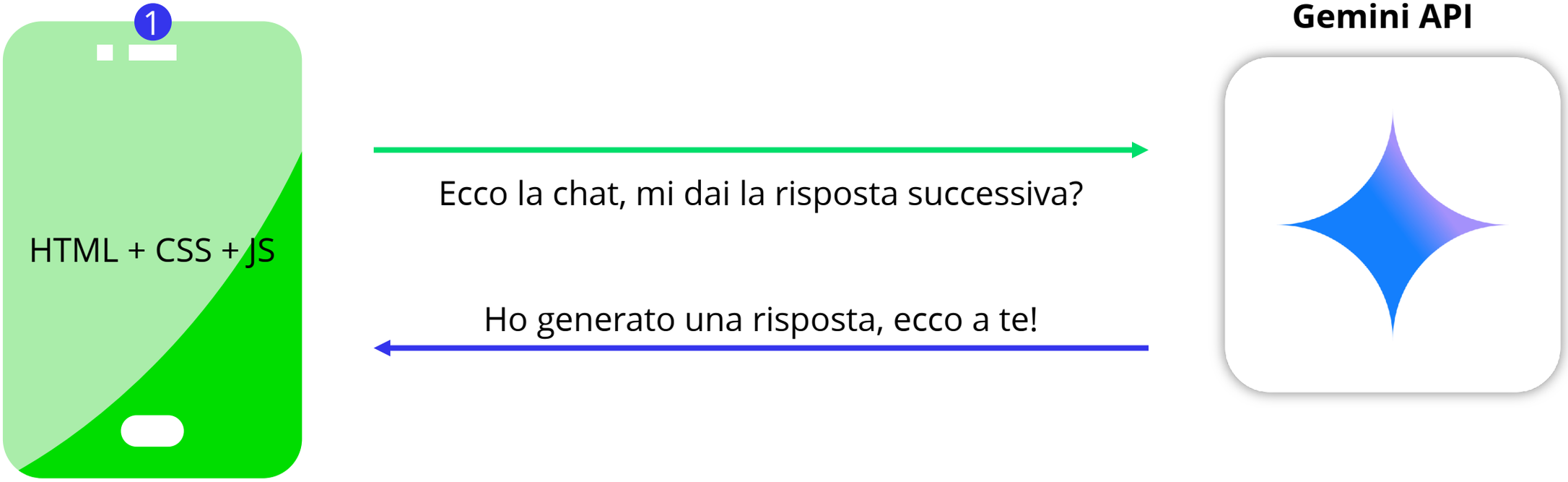


Chiamate AJAX

Eseguire una chiamata che POSTa dati

Il nostro obiettivo

Sfruttare un servizio esterno alla nostra app!



Fetch

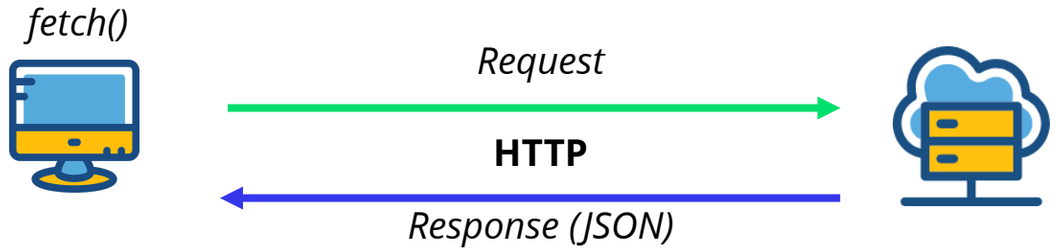
Eseguire chiamate asincrone a un server

Il metodo **Fetch** consente di effettuare **richieste asincrone** verso server web e di gestire le risposte ottenute.

La chiamata andrà rivolta a un indirizzo speccifico detto **endpoint**.

Una volta ricevuta la risposta, dovremo rielaborarla per raccoglierne i dati.

```
1  const endpoint = 'https://api.example.com/data'
2
3  async function getData() {
4      const response = await fetch(endpoint);
5
6    // Resto dell'implementazione
7  }
8
```



JS + HTTP = Ajax!

Async/ await

Se vogliamo che il browser attenda la risposta prima di andare avanti, possiamo usare **await** davanti alla chiamata asincrona.

Per farlo,però, dobbiamo indicare che la funzione è asincrona, premettendo la keyword **async**

```
1  const endpoint = 'https://api.example.com/data'
2
3  async function getData() {
4      const response = await fetch(endpoint);
5
6    // Resto dell'implementazione
7  }
8
```

Facciamo una richiesta

Il secondo argomento di Fetch

Spesso quando facciamo una richiesta dobbiamo anche inviare noi dei dati.

Ad esempio, in questo caso, è necessario mandare la chat a Gemini, in modo che possa generare una risposta sensata!

Il secondo argomento di **fetch** è infatti un oggetto, nel quale inserire diverse informazioni!

```
1 const endpoint = 'http://example.com';
2
3 const response = await fetch(endpoint,
4   {
5     method: '...',
6     headers: { ... },
7     body: JSON.stringify({ contents: ... })
8   }
9 )
```

Facciamo una richiesta

Cosa dobbiamo specificare?

Endpoint

Un'indirizzo da chiamare (sotto forma di *stringa)*

Metodo

Il metodo HTTP con cui effettuare la chiamata (come stringa)

Headers (intestazioni)

Ad esempio il tipo di applicazione, il linguaggio, credenziali, ..., (sotto forma di *oggetto)*

```
'Content-Type': 'application/json'
```

Body

Le informazioni che vogliamo inviare al server (come **stringa JSON**)

```
1 const endpoint = 'http://example.com';
2
3 const response = await fetch(endpoint,
4   {
5     method: '...',
6     headers: { ... },
7     body: JSON.stringify({ contents: ... })
8   }
9 )
```